UTEC | TECNÓLOGO EN INFORMÁTICA

Taller de Sistemas de Información .NET

Laboratorio

Cambio de requerimientos y aclaraciones

Plataforma Multioperador de Distribución de Última Milla

Edición 2026

Docente: Nicolás Escobar

Presentación del cambio	viernes 30 de octubre de 2026
Entrega final	domingo 15 de noviembre de 2026, 23:59 h
Defensa	jueves 19 de noviembre de 2026

Índice

1. Presentación 1

2. Cambio de requerimientos 1

3. Aclaraciones sobre la letra 2

1. Presentación

Tal como se anunció en la letra del laboratorio, en esta instancia se presenta el cambio de requerimientos que cada equipo deberá incorporar a su solución. La entrega final y la defensa mantienen las fechas originales.

El documento tiene dos partes. La primera describe el cambio de requerimientos, que es de incorporación obligatoria. La segunda reúne aclaraciones sobre dos puntos de la letra, el caché distribuido y las banderas de funcionalidad, que fueron objeto de consultas en los monitoreos. Se incluyen por escrito para que la respuesta llegue por igual a todos los equipos y no solo a quienes la plantearon. Estas aclaraciones no amplían el alcance; precisan lo que ya se pedía y orientan sobre las herramientas disponibles.

La letra original se mantiene vigente en todo aquello que este documento no modifica. Donde exista alguna diferencia, prevalece lo indicado aquí.

2. Cambio de requerimientos

El propósito del cambio es el que se planteó desde el inicio del laboratorio: evaluar en qué medida la arquitectura construida permite incorporar requerimientos nuevos sin reescribir lo existente. Los cambios se apoyan en partes del sistema que a esta altura ya deberían estar en funcionamiento, y deben integrarse respetando las mismas exigencias no funcionales de la letra: máquina de estados declarativa, configuración versionada por operador, patrón outbox, idempotencia en el consumidor, operación sin conectividad y aislamiento entre inquilinos.

Los equipos de 4 integrantes deberán incorporar los dos cambios que se describen a continuación. Los equipos de 3 integrantes deberán incorporar únicamente el primero. Los requerimientos funcionales nuevos continúan la numeración de la letra.

2.1 Entrega parcial de bultos

Hasta ahora el envío se trató como una unidad que se entrega completa o no se entrega. En la operación real esto no siempre ocurre así: un envío de tres bultos puede llegar con uno de ellos dañado, el destinatario puede rechazar solo una parte, o alguno de los bultos puede no haberse cargado en el vehículo. A partir de este cambio, cada bulto tiene su propio resultado y puede entregarse, quedar pendiente o devolverse con independencia de los demás bultos del mismo envío.

31. El estado se registra a nivel de bulto, y el estado del envío se deriva de los estados de sus bultos. Se incorpora el estado *entregado parcialmente*. Las transiciones, tanto de bultos como de envíos, siguen definidas de forma declarativa y verificable.
32. En cada parada, el repartidor indica mediante escaneo qué bultos entrega. Para los que no se entregan se registra el motivo, seleccionado del catálogo configurado por el operador. La prueba de entrega queda asociada a los bultos efectivamente entregados. Todo esto debe funcionar sin conectividad, y la política de resolución de conflictos debe contemplar los casos que se producen a nivel de bulto.
33. Cada operador define en sus reglas operativas si admite la entrega parcial y qué sucede con los bultos pendientes: una nueva visita, con o sin recargo, o su devolución al comercio. Estas reglas son configurables y versionadas, igual que el resto.
34. Los bultos pendientes quedan disponibles para ser asignados a una nueva hoja de ruta.

35. El seguimiento público muestra el avance de la entrega (por ejemplo, dos de tres bultos entregados) sin exponer identificadores internos.
36. Los avisos a comercios incorporan los eventos de bulto. Como los sistemas de los comercios ya reciben avisos con el formato actual, el cambio no puede afectarlos, por lo que el contrato de los avisos deberá versionarse.

2.2 Retiro en domicilio (equipos de 4 integrantes)

Los comercios necesitan resolver los cambios y devoluciones de sus compradores sin que estos deban trasladarse. Se incorpora para ello el retiro en domicilio: a pedido del comercio, el operador concurre al domicilio del comprador, retira uno o más bultos y los hace llegar al comercio. No debe confundirse con la devolución que ya contempla la letra, que corresponde a envíos que no pudieron entregarse. En el retiro, el flujo comienza en el comprador y termina en el comercio.

37. El comercio solicita el retiro desde el portal y por API, indicando los bultos, el domicilio, la franja horaria y el motivo.
38. El retiro tiene su propio ciclo de vida dentro de la máquina de estados y convive con el de los envíos. A modo de referencia, sus estados podrían ser: solicitado, asignado a ruta, retirado, no retirado, en depósito y entregado al comercio.
39. Las hojas de ruta pasan a ser mixtas, con paradas de entrega y paradas de retiro. La validación de capacidad del vehículo debe considerar cómo varía la carga a lo largo del recorrido, ya que los bultos retirados ocupan lugar desde la parada en que se suben.
40. La aplicación del repartidor registra el retiro con la identificación del bulto, una fotografía de su estado y la firma de quien lo entrega, también sin conectividad.
41. El retiro tiene su propia tarifa dentro del cuadro tarifario. El comprador puede seguirlo mediante el mismo enlace público que se utiliza para los envíos, y tanto los avisos a comercios como el tablero de operación y los reportes deben incorporarlo.

2.3 Verificación

El cambio deberá demostrarse en ejecución en el monitoreo del jueves 12 de noviembre. Además, se agregan dos escenarios a los ya anunciados para la defensa. En el primero se entregará parcialmente un envío sin conectividad, se sincronizará el dispositivo y se verificará el resultado en el backoffice, en el seguimiento público y en el aviso recibido por el comercio. En el segundo, solo para los equipos de 4 integrantes, se ejecutará una hoja de ruta mixta con entregas y retiros.

Cada equipo deberá incluir, entre sus registros de decisión de arquitectura, uno dedicado al cambio, en el que analice qué componentes tuvo que modificar, qué resultó sencillo de incorporar, qué no lo fue y por qué. Este registro puede contarse dentro de los ocho exigidos.

3. Aclaraciones sobre la letra

Durante los monitoreos, varios equipos consultaron qué se espera exactamente en relación con el caché distribuido y con las banderas de funcionalidad. Las respuestas se dieron en su momento a cada equipo, pero como las dudas se repitieron, se considera conveniente dejarlas por escrito para que todos trabajen con el mismo criterio, que es además el que se aplicará en la evaluación.

3.1 Caché distribuido

La letra exige caché *distribuido*, y la distinción es relevante por la forma en que se despliega la solución. Un caché en memoria, como el que ofrece `IMemoryCache`, vive dentro de cada proceso. Si la API se ejecuta en dos instancias detrás del balanceador, cada una mantiene su propia copia de los datos. Cuando el administrador modifica una tarifa y su solicitud la atiende una de las instancias, esa instancia puede descartar su copia, pero la otra no se entera y sigue respondiendo con el valor anterior hasta que la entrada expire. Por otra parte, la letra pide una API sin estado en memoria, y un caché local es justamente estado en memoria.

El caché distribuido resuelve este problema ubicando los datos en un almacén externo, compartido por todas las instancias. La opción más difundida es Redis, que la mayoría de los equipos

ya incluye en su `docker compose`. En .NET, la abstracción `IDistributedCache` permite trabajar sobre Redis, y en versiones recientes `HybridCache` combina una copia local en memoria con un almacén distribuido. Quien opte por un esquema de dos niveles deberá resolver cómo se enteran las demás instancias de una invalidación; bibliotecas como FusionCache contemplan esa situación. La elección queda a cargo del equipo y debe fundamentarse en el registro de decisión sobre caché e invalidación.

La letra pide al menos dos casos de uso con perfiles de acceso diferentes, y sugiere el seguimiento público y la configuración tarifaria. Vale la pena detenerse en por qué son distintos. El seguimiento público recibe un volumen de consultas muy alto, pero el estado de un envío cambia varias veces a lo largo del día; resulta razonable una expiración corta y, sobre todo, la invalidación de la entrada en el momento en que se procesa un evento de ese envío. La configuración tarifaria también se consulta constantemente, pero cambia muy poco, y como la letra exige que esté versionada, una versión ya publicada no vuelve a modificarse. Eso simplifica bastante el problema de la invalidación. Se espera que cada equipo razone de esta manera la política de expiración e invalidación de cada caso y la deje documentada.

En la evaluación se tendrá en cuenta, además, lo siguiente:

- Las entradas del caché deben distinguir al operador, de modo que el caché no se convierta en una vía de filtración de datos entre inquilinos.
- La métrica de aciertos y fallos debe poder consultarse por caso de uso en el tablero técnico. Las abstracciones de caché de .NET no la proveen, de modo que debe instrumentarse, por ejemplo, con las métricas de OpenTelemetry que ya exige la letra.
- Si el caché no está disponible, el sistema debe seguir funcionando contra la base de datos, aunque con un rendimiento menor.
- Los equipos de 3 integrantes, aunque estén eximidos del balanceo, deben implementar igualmente caché distribuido.

La demostración esperada consiste en modificar una tarifa a través de una de las instancias de la API y verificar que la otra instancia responde de inmediato con el valor nuevo.

Por último, el mismo razonamiento alcanza a otros mecanismos que la letra ya exige y que dependen del estado compartido entre instancias: el canal de SignalR, la protección de datos de las cookies, la ejecución única de las tareas programadas y la limitación de tasa. En particular, el limitador que incluye ASP.NET Core lleva la cuenta en cada instancia, de modo que con dos instancias el límite efectivo resulta el doble del configurado.

3.2 Banderas de funcionalidad

El requerimiento opcional de banderas de funcionalidad puede confundirse con la configurabilidad que la letra ya exige como obligatoria, por lo que conviene distinguir ambos conceptos. Las reglas operativas y el cuadro tarifario ajustan el comportamiento de algo que existe: cuántos intentos de entrega se admiten o cuánto cuesta un envío urgente. Una bandera de funcionalidad determina, en cambio, si una funcionalidad está disponible o no para un operador. Por ejemplo, que el operador A ofrezca a sus destinatarios la reprogramación desde el seguimiento público y que el operador B no lo haga.

En la industria las banderas se utilizan para ofrecer planes con distintas prestaciones, para habilitar una funcionalidad nueva en un cliente antes que en el resto, o para desactivar con rapidez algo que presenta fallas sin necesidad de un nuevo despliegue. El propio cambio de requerimientos de este documento es un buen ejemplo de aplicación: un equipo que implemente banderas podría habilitar la entrega parcial o el retiro en domicilio operador por operador.

Se entiende que la activación y desactivación de las banderas corresponde a un administrador de la plataforma, es decir, a quien ofrece el servicio, y no a los usuarios de cada operador. En el ecosistema .NET existe la biblioteca `Microsoft.FeatureManagement`, que admite filtros personalizados y puede adaptarse al criterio de inquilino. También se aceptará una implementación propia, siempre que se justifique.

Para otorgar los puntos de este requerimiento se verificará que:

- existan al menos dos funcionalidades reales bajo bandera, activables por operador desde una interfaz de administración;
- la bandera se respete en todas las capas en que la funcionalidad interviene: la API, las aplicaciones web, el worker y la aplicación del repartidor;
- un cambio de bandera alcance a todas las instancias de la API en un tiempo acotado y conocido, sin necesidad de reiniciarlas;
- quede registro de quién modificó cada bandera y en qué momento.